

Relatorio Tecnico - Viva Mulher

1. O Backend e o Banco de Dados do Projeto

Ao analisar a estrutura do projeto, percebe-se que ele é uma aplicação Front-end pura construída com HTML, CSS e JavaScript.

O projeto não possui um Backend clássico (como um servidor rodando Node.js, Python, PHP ou Java) nem um banco de dados hospedado em nuvem (como MySQL, PostgreSQL ou MongoDB). Toda a lógica de negócio (como login, cadastro e agendamentos) acontece diretamente no navegador do usuário.

Onde está localizado o Banco de Dados?

O 'banco de dados' do sistema é, na verdade, o LocalStorage, uma funcionalidade nativa do HTML5 (Web Storage API). O LocalStorage permite que aplicações web armazenem dados diretamente no navegador do usuário, persistindo as informações mesmo quando a aba ou o navegador são fechados.

No código, os dados são convertidos em texto (formato JSON) e salvos no navegador da seguinte forma:

```
localStorage.setItem('vivaMulherUsuarios', JSON.stringify(usuarios));
```

Como acessar o Banco de Dados?

Para visualizar, editar ou apagar os dados cadastrados, tudo é feito pelo próprio navegador (Google Chrome, Edge, Firefox):

1. Abra a página do projeto no navegador.
2. Pressione a tecla F12 no teclado (ou Inspeccionar).
3. Clique na aba Application (Aplicativo).
4. No menu lateral, expanda a seção Storage -> Local Storage e selecione o site.
5. À direita aparecerão as chaves 'vivaMulherUsuarios' (todos cadastrados) e 'vivaMulherUsuarioAtivo'.

2. Mascaramento (Ofuscação) do Código JavaScript

Para esconder a lógica do sistema e evitar cópias fáceis, o arquivo script.js utilizou uma técnica de Ofuscação combinada com Codificação Base64.

Relatorio Tecnico - Viva Mulher

A versao legivel (script_dev.js) foi comprimida e codificada. A tecnica utilizou as seguintes etapas:

1. Conversao para Base64 (btoa): Todo o texto legivel foi convertido numa string continua de Base64.
2. Camuflagem de Funcoes Nativas: O codigo usou um vetor para esconder funcoes que fariam a descriptografia.

Como o Navegador le isso?

1. O Chrome executa a funcao atob() para reverter a string codificada de volta para texto.
2. Aplica funcoes de escape (decodeURIComponent) para os acentos e formatacoes.
3. Por ultimo, o codigo e envolvido na funcao eval(), que executa todo o texto decodificado como JavaScript em memoria.

Apesar de dificultar a copia, o uso de eval() com Base64 e considerado fraco em ciberseguranca pois pode ser facilmente revertido desfazendo as etapas e substituindo o eval() por um console.log() para imprimir o codigo na tela.